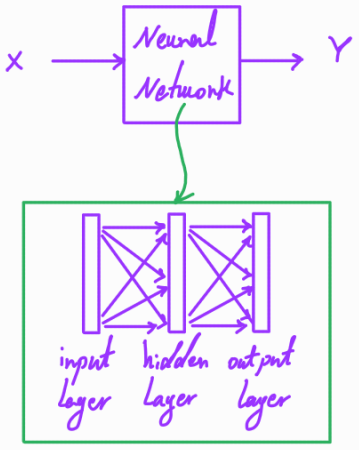
Overview of RL Algorithms

- So far, we have been dealing with planning settings where the full environment model (transition function P and reward function R) is known.
  - value iteration
  - policy iteration
  - generalized policy iteration
  - linear programming

- Now, we move to learning settings where the environment model is not fully known.
  - model-based RL: $D \rightarrow \hat{P}, \hat{R} \rightarrow \hat{\pi}^*$; $\hat{P} \approx P$, $\hat{R} \approx R$
  - model-free RL: $D \rightarrow \hat{\pi}^*$
    - Monte-Carlo
    - Temporal difference
    - TD(λ)

Function Approximation



Functions of interest in RL

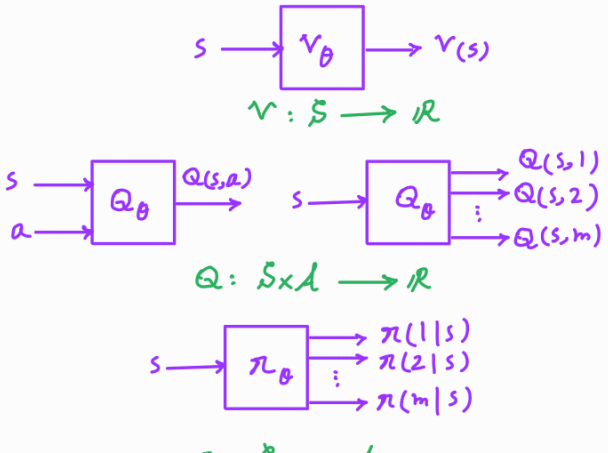
value functions V, Q

V^*, Q^*

Policy π / π^*

Environment model $\hat{P}$, $\hat{R}$

RL with NN approximators (Deep RL)



Monte Carlo Method

Rather than having the full environment model (transition function P and reward function R), Monte Carlo methods only require experiences, i.e., sample trajectories (sequence of states, actions, rewards).

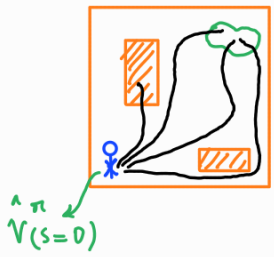
Model_free method

Monte Carlo: Using random sampling to perform a computational task, such as estimation and optimization.

Assumption: The setting is episodic, i.e., the interactions happen in episodes of finite length.

[The updates of MC-based methods happen after each episode
⟶ not fully online]

MC for policy evaluation: Given a stationary policy π, we want to compute

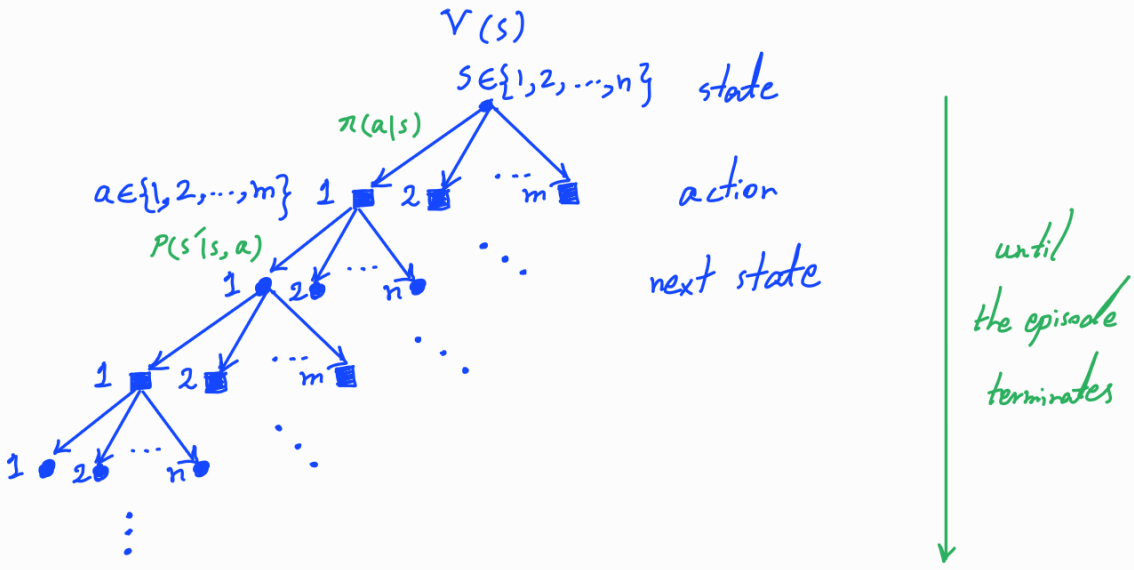


$$V^{\pi}(s) = \mathop{\mathbb{E}}_{\substack{A_t \sim \pi \\ s_{t+1} \sim P(\cdot \mid s_t, A_t)}} \Big[\underbrace{\sum_{t=0}^{T} \gamma^t R(s_t, A_t)}_{\text{return } G} \mid s_0 = s \Big]$$

Idea: Estimate the expected return using the empirical mean of the returns of sample trajectories.

$$\mathbb{E}_{P_X}[X] \approx \hat{\mathbb{E}}[X] = \frac{1}{N} \sum_{i=1}^{N} x_i$$

x can be return G ← x_i sampled from P_X



Let $\mathcal{T} = \{\tau_1, \tau_2, \dots, \tau_{|\mathcal{T}|}\}$ denote a set of trajectories sampled from s

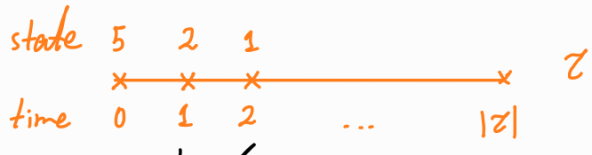
$$\longrightarrow V^{\pi}(s) = \mathop{\mathbb{E}}_{\substack{A_t \sim \pi \\ s_{t+1} \sim P(\cdot \mid s_t, A_t)}} \Big[\sum_{t=0}^{T} \gamma^t R(s_t, A_t) \mid s_0 = s \Big] \approx \frac{1}{|\mathcal{T}|} \Big(\sum_{\tau \in \mathcal{T}} \underbrace{\sum_{t=0}^{|\tau_i|} \gamma^t R(s_t^i, a_t^i)}_{\text{return for trajectory } \tau_i} \Big) = \hat{V}^{\pi}(s)$$

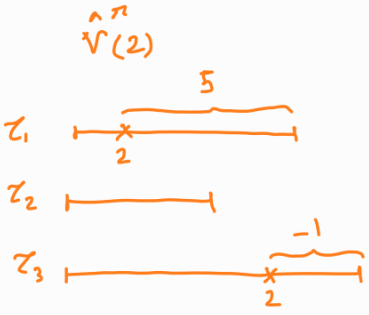
i.i.d. samples

First-visit MC method

Iterative method:

- Initialize $\hat{V}^\pi$
- Initialize $G(s) = \emptyset$ for all $s \in S$
- Repeat
  - Generate a sample trajectory τ using π
  - For $s \in S$ s.t. $s \in \tau$:





  - t_s = first time s is visited
  - $G = \sum_{t=t_s}^{|\tau|} \gamma^{t-t_s} R(s_t, a_t)$
  - $G(s) \leftarrow G(s) \cup \{G\}$
  - $N(s) \leftarrow N(s) + 1$ $\quad N(s) = |G(s)|$
  - $\hat{V}^\pi(s) = \frac{1}{N(s)} \sum_{G \in G(s)} G$

$$\hat{\mu}_{k+1} = \frac{\sum_{i=1}^{k+1} x_i}{k+1} = \frac{\sum_{i=1}^{k} x_i + x_{k+1}}{k+1} = \frac{k\hat{\mu}_k + x_{k+1}}{k+1}$$

$$= \frac{k}{k+1}\hat{\mu}_k + \frac{1}{k+1}x_{k+1} = \hat{\mu}_k + \frac{1}{k+1}(x_{k+1} - \hat{\mu}_k)$$

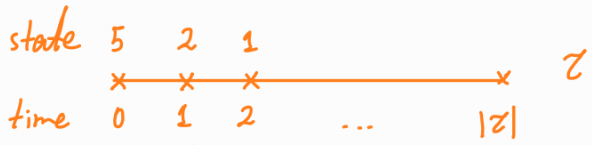
Incremental computation of mean

Note: By Law of large numbers, $\hat{V}^\pi(s) \xrightarrow[N(s)\to\infty]{} V^\pi(s)$.

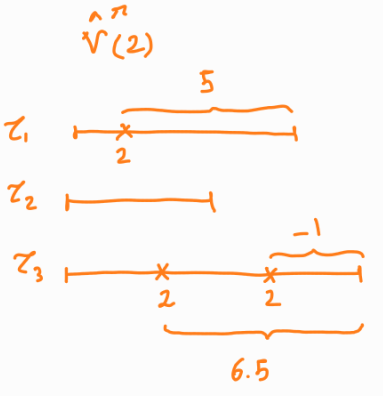
Every-visit MC method

Iterative method:

- Initialize $\hat{V}^{\pi}$
- Initialize $G(s) = \emptyset$ for all $s \in S$
- Repeat
  - Generate a sample trajectory τ using π
  - For $s \in S$ s.t. $s \in \tau$:



    - For every time s is visited:



$$\hat{V}^{\pi}(2) \approx \frac{5 + 6.5 + (-1)}{3}$$

- t_s = time s is visited
- $G = \sum_{t=t_s}^{|\tau|} \gamma^{t-t_s} R(s_t, a_t)$
- $G(s) \leftarrow G(s) \cup \{G\}$
- $N(s) \leftarrow N(s) + 1$
- $\hat{V}^{\pi}(s) = \frac{1}{N(s)} \sum_{G \in G(s)} G$

Note: It can be shown that $\hat{V}^{\pi}(s) \xrightarrow[N(s) \to \infty]{} V^{\pi}(s)$.

MC for policy optimization: We want to find an (approximately) optimal policy π^* through iterative policy evaluation and policy improvement.

Idea: → generalized policy iteration

- Evaluate the policy by estimating the Q function using the empirical mean of the returns of the sample trajectories.
- Improve the policy using the estimated Q function in a greedy manner.

Estimating V or Q ?

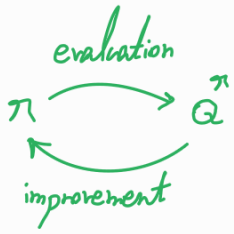
If V known $\longrightarrow \pi = \underset{a \in A}{\operatorname{argmax}} \left[\overbrace{R(s,a) + \gamma \mathbb{E}_{s' \sim P(\cdot|s,a)} [V(s')]}^{Q(s,a)} \right]$ (unknown)

If Q known $\longrightarrow \pi = \underset{a \in A}{\operatorname{argmax}} \; Q(s,a)$

$\longrightarrow$ we need to estimate Q instead of V.

MC method with exploring starts

Iterative method:



- Initialize $\hat{Q}$
- Initialize π
- Initialize $G(s,a) = \emptyset$ for all $s \in S$, $a \in A$
- Exploration: Choose $d \in \Delta(S \times A)$ s.t. $d(s,a) > 0$ for all $s \in S$, $a \in A$
  - $\Delta(S \times A)$: set of all possible probability distributions over $S \times A$
- Repeat
  - Sample $s_0 \in S$ and $a_0 \in A$ according to d
  - Generate a sample trajectory τ using π starting from (s_0, a_0)
  - policy evaluation:
    - For $(s,a) \in S \times A$ s.t. $(s,a) \in \tau$:
      - $t_{s,a}$ = first time (s,a) is visited
      - $G = \sum_{t=t_{s,a}}^{|\tau|} \gamma^{t-t_{s,a}} R(s_t, a_t)$
      - $G(s,a) \leftarrow G(s,a) \cup \{G\}$
        - $G(s=1, a=3) = \{-5, 7, 1.2, \dots\}$
      - $N(s,a) \leftarrow N(s,a) + 1$
        - $N(s,a) = |G(s,a)|$
      - $\hat{Q}(s,a) = \frac{1}{N(s,a)} \sum_{G \in G(s,a)} G$

Policy improvement:

- For $s \in \mathcal{S}$ s.t. $s \in \mathcal{T}$:
  - $\pi(s) = \underset{a \in \mathcal{A}}{\operatorname{argmax}} \ \hat{Q}(s,a)$

Note: In real-world settings, it may not be possible to start from any state-action pairs.

⟶ Can we instead encourage continuous exploration?

Idea:

- Use a soft (stochastic) policy $\pi(a|s)$ such that it chooses all (possible) actions at each state with a non-zero probability, i.e.,

$$\pi(a|s) > 0 \quad \forall s \in \mathcal{S}, \forall a \in \mathcal{A}.$$

- As the estimates improve over time, reduce exploration.

MC method with ϵ-greedy exploration

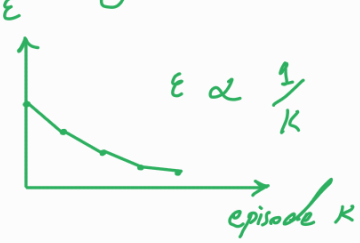
Iterative method:

- Initialize $\hat{Q}$
- Initialize $\pi(a|s)$ s.t. $\pi(a|s) > 0$ for all $s \in S$, $a \in A$
- Initialize $G(s,a) = \emptyset$ for all $s \in S$, $a \in A$
- Repeat
  - Generate a sample trajectory τ using π

policy evaluation

  - For $(s,a) \in S \times A$ s.t. $(s,a) \in \tau$:
    - $t_{s,a}$ = first time (s,a) is visited
    - $G = \sum_{t=t_{s,a}}^{|\tau|} \gamma^{t-t_{s,a}} R(s_t, a_t)$
    - $G(s,a) \leftarrow G(s,a) \cup \{G\}$
    - $N(s,a) \leftarrow N(s,a) + 1$
    - $\hat{Q}(s,a) = \frac{1}{N(s,a)} \sum_{G \in G(s,a)} G$

Decaying exploration:



Policy improvement

- For $s \in \mathcal{S}$ s.t. $s \in \mathcal{T}$:
  - $a^* = \underset{a \in \mathcal{A}}{\operatorname{argmax}} \hat{Q}(s,a)$
  - For $a \in \mathcal{A}$:
    - $\pi(a|s) = \begin{cases} 1 - \varepsilon + \frac{\varepsilon}{|\mathcal{A}|} & \text{if } a = a^* \\ \frac{\varepsilon}{|\mathcal{A}|} & \text{otherwise} \end{cases}$

$\pi(a|s)$ → ε-greedy

$\frac{\varepsilon}{|\mathcal{A}|}$ → Exploration
